NEW

Example in Oracle

```
CUSTOMER (
        CustomerID NUMBER PRIMARY KEY,
        Name VARCHAR2(100)
);

ORDERS (
        OrderID NUMBER PRIMARY KEY,
        CustomerID NUMBER,
        OrderDate DATE,
        Amount NUMBER,
        FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
);
```

CUSTOMER: 100K tuples
ORDERS: several million tuples

```
EXPLAIN PLAN FOR
SELECT c.Name, o.OrderID, o.OrderDate, o.Amount
FROM Customer c JOIN Orders o ON o.CustomerID = c.CustomerID
WHERE c.CustomerID = 42;
```

NEW

Example in Oracle

```
CUSTOMER (
        CustomerID NUMBER PRIMARY KEY,
        Name VARCHAR2(100)
);

ORDERS (
        OrderID NUMBER PRIMARY KEY,
        CustomerID NUMBER,
        OrderDate DATE,
        Amount NUMBER,
        FOREIGN KEY (CustomerID)
            REFERENCES CUSTOMER(CustomerID)
);
```

```
EXPLAIN PLAN FOR
SELECT c.Name, o.OrderID, o.OrderDate, o.Amount
FROM Customer c JOIN Orders o ON
        o.CustomerID = c.CustomerID
WHERE c.CustomerID = 42;
```

```
-------------------------------------------------------------------------------------------
| Id | Operation                    | Name         | Rows | Bytes | Cost (%CPU)| Time |
-------------------------------------------------------------------------------------------
|  0 | SELECT STATEMENT             |              |   50 |  2500 |  8450   (1)|      |
|  1 |  NESTED LOOPS                |              |   50 |  2500 |  8450   (1)|      |
|  2 |   TABLE ACCESS BY INDEX ROWID| CUSTOMER     |    1 |    30 |     2   (0)|      |
|* 3 |    INDEX UNIQUE SCAN         | SYS_C...     |    1 |       |     1   (0)|      |
|* 4 |   TABLE ACCESS FULL          | ORDERS       |   50 |  1000 |  8448   (1)|      |
-------------------------------------------------------------------------------------------

Predicate Information:
---------------------------------------------------
   3 - access("C"."CUSTOMERID"=42)
   4 - filter("O"."CUSTOMERID"=42)
```

Scan ORDERS
Keep orders having CustomerID = 42
Find CustomerID = 42 in the index
(primary key)
Retrieve the CUSTOMER tuple
Join the results
Return SELECT result

NEW

Same Result, Different Access Paths

```
CUSTOMER (
        CustomerID NUMBER PRIMARY KEY,
        Name VARCHAR2(100)
);

ORDERS (
        OrderID NUMBER PRIMARY KEY,
        CustomerID NUMBER,
        OrderDate DATE,
        Amount NUMBER,
        FOREIGN KEY (CustomerID)
            REFERENCES CUSTOMER(CustomerID)
);
```

```
EXPLAIN PLAN FOR
SELECT c.Name, o.OrderID, o.OrderDate, o.Amount
FROM Customer c JOIN Orders o ON
        o.CustomerID = c.CustomerID
WHERE c.CustomerID = 42;
```

```
CREATE INDEX idx_orders_customerON
Orders(CustomerID);
```

```
-----------------------------------------------------------------------------------------------
| Id | Operation                            | Name                | Rows | Bytes | Cost (%CPU)|
-----------------------------------------------------------------------------------------------
|  0 | SELECT STATEMENT                     |                     |   50 |  2500 |    55   (0)|
|  1 |  NESTED LOOPS                        |                     |   50 |  2500 |    55   (0)|
|  2 |   TABLE ACCESS BY INDEX ROWID        | CUSTOMER            |    1 |    30 |     2   (0)|
|* 3 |    INDEX UNIQUE SCAN                 | SYS_C...            |    1 |       |     1   (0)|
|  4 |   TABLE ACCESS BY INDEX ROWID BATCHED| ORDERS              |   50 |  1000 |    53   (0)|
|* 5 |    INDEX RANGE SCAN                  | IDX_ORDERS_CUSTOMER |   50 |       |     3   (0)|
-----------------------------------------------------------------------------------------------

Predicate Information:
----------------------------------------------------
   3 - access("C"."CUSTOMERID"=42)
   5 - access("O"."CUSTOMERID"=42)
```

Find CustomerID = 42 in the
ORDERS index
(IDX_ORDERS_CUSTOMER)
Retrieve the matching ORDERS tuples
…

NEW

Query Formulation Matters!

```
CREATE INDEX idx_orders_dateON
Orders(OrderDate);
```

```
SELECT *
FROM Orders
WHERE EXTRACT(YEAR FROM OrderDate)
        = 2026;
```

```
-------------------------------------------------
| Id | Operation         | Name   | Rows | Bytes |
-------------------------------------------------
|  0 | SELECT STATEMENT  |        | 5000 |  150K |
|* 1 |  TABLE ACCESS FULL| ORDERS | 5000 |  150K |
-------------------------------------------------

Predicate Information:
-------------------------------------------------
   1 - filter(EXTRACT(YEAR FROM "ORDERDATE")=2026)
```

```
SELECT *
FROM Orders
WHERE OrderDate >= DATE '2026-01-01'
        AND OrderDate <  DATE '2027-01-01';
```

```
-----------------------------------------------------------------
| Id | Operation                           | Name            |
-----------------------------------------------------------------
|  0 | SELECT STATEMENT                    |                 |
|  1 |  TABLE ACCESS BY INDEX ROWID BATCHED| ORDERS          |
|* 2 |   INDEX RANGE SCAN                  | IDX_ORDERS_DATE |
-----------------------------------------------------------------

Predicate Information:
-------------------------------------------------
   2 - access("ORDERDATE">=DATE '2026-01-01'
            AND "ORDERDATE"<DATE '2027-01-01')
```

NEW

Also selectivity matters!

CREATE INDEX idx_orders_dateON Orders(OrderDate);

```
SELECT *
FROM Orders
WHERE OrderDate >= DATE '2026-01-01'
        AND OrderDate <  DATE '2027-01-01';
```

```
-----------------------------------------------------
| Id | Operation         | Name   | Rows    | Bytes |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |        | 8000000 |       |
|* 1 |  TABLE ACCESS FULL| ORDERS | 8000000 |       |
-----------------------------------------------------

Predicate Information:
-----------------------------------------------------
   1 - filter("ORDERDATE">=DATE '2026-01-01'
              AND "ORDERDATE"<DATE '2027-01-01')
```

E.g., Oracle estimates that **80% of ORDERS** satisfy the condition

Good statistics → good cardinality estimates → potentially better execution plans